الفصل الرابع ASE

الطبقة العميقة (Tier 2) — Software Project Planning

أسئلة تحليل/تقييم/حساب مفصّل خطوة-خطوة — الحل الكامل غير الموجود بالسلايدات

شنو هذا الملف؟ الطبقة اللي **الدكتور ما يشرحها**. أنواعها: فهم (Comprehension) · تحليل (Analyze) · تقييم (Evaluate) · تصميم (Create) · **حساب (Calc)** ← البطل. **قاعدة الحل:** كل مسألة حسابية تُعرض بالقالب المفصّل: معطيات ← معادلة ← تعويض خطوة-خطوة ← نتيجة ← تحقق ← خطأ شائع. **المصدر:** نص الفصل + الأمثلة المحلولة 4.1–4.19 (مدمجة بالسلايدات).

المحور 1 — الحساب (Calc) ★★★ بطل الفصل

1.A — Function Points (FP / CAF)

مثال محلول (مرجعي): Example 4.1 (سلايد 23–24)

المعطيات: 50=user inputs · 40=user outputs · 35=user inquiries · 6=(ILF)files · 4=(EIF)ext.interfaces — كلها average. **المعادلة:** FP = UFP · UFP = Σ(count × weight) · CAF = 0.65 + 0.01 × ΣFi

التعويض بالتدريج:

الوحدة	العدد	الوزن (average)	الناتج
EI (inputs)	50	4	200
EO (outputs)	40	5	200
EQ (inquiries)	35	4	140
ILF (files)	6	10	60
EIF (ext.interfaces)	4	7	28
UFP			**628**

ΣFi = 14 × 3 = 42 حيث (كل عامل = 3 moderate)

CAF = 0.65 + 0.01 × ΣFi

= 0.65 + 0.42 = 1.07

FP = 628 × 1.07 = 671.96 ≈ 672

التحقق: CAF بين 0.65 و1.35 ⇒ 1.07 منطقي. ✓ **الخطأ الشائع:** نسيان ضرب ΣFi بـ14 (14 عامل، مو واحد).

مثال محلول: Example 4.2 (سلايد 25–26) — CAF معطى جاهزاً

المعطيات: low EI 10 · high EO 12 · low ILF 20 · high EIF 15 · average EQ 12 · **CAF معطى = 1.10**. **التعويض:**

الوحدة	العدد	الوزن	الناتج
EI	10	3 (low)	30
EO	12	7 (high)	84
ILF	20	7 (low)	140
EIF	15	10 (high)	150
EQ	12	4 (avg)	48
UFP			**452**

FP = UFP × CAF = 452 × 1.10 = 497.2

الدرس: أحياناً السؤال يعطيك **CAF جاهز** بدل ΣFi — لا تتوه بحساب العوامل، عوّض مباشرة. *(هذا هو الفرق بين 4.1 و4.2).*

مثال محلول: Example 4.3 (سلايد 27–30) — تعقيدات مختلطة

المعطيات: .low EI: 10 low, 15 avg, 17 high · EO: 6 low, 13 high · EQ: 3 low, 4 avg, 2 high · ILF: 2 avg, 1 high · EIF: 9
التعويض: | الوحدة | التفصيل | الحساب | المجموع | |---|---|---|---| | EI | 10×3 + 15×4 + 17×6 | 30+60+102 | **192** | | EO | 6×4 + 13×7 | 24+91 | **115** | | EQ | 3×3 + 4×4 + 2×6 | 9+16+12 | **37** | | ILF | 2×10 + 1×15 | 20+15 | **35** | | EIF | 9×5 | 45 | **45** | | **UFP** | | | **424** |

ΣFi (المعدّلة): (3) significant data comm + (5) performance very critical + (3) moderately reusable

- 41 = (30 = 10 × 3) average الباقي + (0) not multi-install

CAF = 0.65 + 0.01×41 = 1.06

FP = 424 × 1.06 = 449.44 ≈ 449

التحقق: ΣFi=41 منطقي (المتوسط = 42 لو كل شي moderate). ✓

1.B — Empirical Models (Walston-Felix & SEL)

مثال محلول: Example 4.4 (سلايد 37–40)

المعطيات: effort = 8 PY = **96 person-months**. المطلوب: LOC, duration, productivity, average manning. **المعادلات (عكسية):** الوزن W-F: L = (E/5.2)^(1/0.91) ⇒ E = 5.2·L^0.91 [L بـKLOC]

الوزن SEL: L = (E/1.4)^(1/0.93) ⇒ E = 1.4·L^0.93

D(W-F) = 4.1·L^0.36 · D(SEL) = 4.6·L^0.26

P = LOC/effort(PM) · M_avg = effort/D

التعويض:

L(SEL) = (96/1.4)^(1/0.93) = (68.57)^1.0753 = 94.264 KLOC = 94,264 LOC

L(W-F) = (96/5.2)^(1/0.91) = (18.46)^1.0989 = 24.632 KLOC = 24,632 LOC

D(W-F) = 4.1 × (24.632)^0.36 = 4.1 × 3.17 = 13 months

D(SEL) = 4.6 × (94.264)^0.26 = 4.6 × 3.26 = 15 months

P(SEL) = 94,264/(8×12) = 94,264/96 = 982 LOC/PM (≈11,783 LOC/PY)

P(W-F) = 24,632/96 = 257 LOC/PM (≈3,079 LOC/PY)

M(SEL) = 96/15 = 6.4 persons · M(W-F) = 96/13 = 7.4 persons

التحقق: SEL يعطي LOC أكثر بكثير لنفس الجهد (a أصغر ⇒ أتفاؤل). ✓ **الخطأ الشائع:** نسيان تحويل PY←PM (×12) قبل التعويض.

1.C — Basic COCOMO

مثال محلول: Example 4.5 (سلايد 47–49) — 400 KLOC بكل المودات

المعادلات: E = a(KLOC)^b | c_b | b_b | a_b | Mode | **التعويض (KLOC=400):** d_b^E = a_b(KLOC)^b_b · D = c_b(E) | d_b | D = c(E)^d | |---|---|---|---|---|---|---| | Organic | 2.4 | 1.05 | 2.4×(400)^1.05 = 2.4×539.71 = **1295.31 PM** | 2.5 | 0.38 | 2.5×(1295.31)^0.38 = **38.07 mo** | | Semidetached | 3.0 | 1.12 | 3.0×(400)^1.12 = 3.0×820.93 = **2462.80 PM** | 2.5 | 0.35 | 2.5×(2462.80)^0.35 = **38.45 mo** | | Embedded | 3.6 | 1.20 | 3.6×(400)^1.20 = 3.6×1325.78 = **4772.81 PM** | 2.5 | 0.32 | 2.5×(4772.81)^0.32 = **37.60 mo** | **التحقق:** كلما زادت "قسوة" الموديل، الجهد يقفز (embedded ≈ 3.7× organic) لكن الوقت يزيد بشكل أبطأ. ✓

مثال محلول: Example 4.6 (سلايد 50–52) — اختيار الموديل

المعطيات: KLOC 200، خبرة فريق متوسطة، جدول متوسط ⇒ **semidetached**.

200^1.12 = 377.7057 ≈ 377.71

E = 3.0 × (200)^1.12 = 3.0 × 377.71 = 1133.12 ≈ 1133 PM

D = 2.5 × (1133.12)^0.35 = 2.5 × 11.72 = 29.30 ≈ 29 months

SS = E/D = 1133.12/29.30 ≈ 38.7 ≈ 39 persons

الفائدة: هذا مثال "كيف تختار الموديل" — الحجم + الخبرة + الجدول.

1.D — Intermediate COCOMO (EAF)

مثال محلول: Example 4.7 (سلايد 65–67) — حالتان بختلف الـEAF

المعطيات: KLOC 400 **embedded** (intermediate model). **Case I — مطوّرون very highly capable:**

EAF = RELY × ... (من جدول الـdriver 15)

= عدد من المضاعفات < 1 (قدرة عالية تخفّض الجهد)

E = a_i(KLOC)^b_i × EAF = 2.8 × (400)^1.20 × EAF

Case II — قدرة منخفضة لكن خبرة عالية باللغة: EAF أكبر (PCAP منخفض يرفع الجهد، لكن LEXP عالية تخفضه جزئياً)

EAF × E = 2.8 × (400)^1.20(أكبر)

الدرس: نفس الحجم والموديل ⇒ الـEAF هو الفرق. **الـdriver 15 هي أداة الضبط الدقيق.**

مثال محلول: Example — Full screen editor (سلايد 68–72)

المعطيات: 5 وحدات (4+2+1+2+3) = 12 KLOC ⇒ **organic**. drivers: ACAP high · (1.15) CPLX high · (1.15) RELY high · (0.86) LEXP low · (1.07) الباقي nominal.

EAF = 1.15 × 1.15 × 0.86 × 1.07 = 1.2169

E = a_i(KLOC)^b_i × EAF = 3.2 × (12)^1.05 × 1.2169 = 52.91 PM

D = c_i(E)^d_i = 2.5 × (52.91)^0.38 = 11.29 months

Organic Small — phase-wise (Table 7، سلايد 72): 16 %system design (PM 8.465) · 26 %detailed design (PM 13.756) · 42 %module code & integration (22.222 PM) · 16 %test (8.465 PM). *(التوزيع للوقت: 0.24/0.19/0.../ من D).*

الدرس: تقدير الجهد **لكل مرحلة** (phase-wise) مو فقط الكلي — سلايد 71–72.

1.E — COCOMO II (Early Design / Post-Architecture)

مثال محلول: Example 4.10 (سلايد 103–105)

المعطيات: application generator، KLOC 50، scale factors: precedentness low, flexibility high dev, team low. Drivers: nominal. PDIF high, PERS high، الباقي nominal. **المعادلة:** $B = 0.91 + 0.01 \times \Sigma(\text{rating})$ · $PM_{nominal} = 2.5 \times (size)^B$ · $PM_{adj} = PM_{nom} \times \Pi(EM)$ Σrating = 4.96 (precedentness) + 2.03 (flexibility) + 4.24 (arch/risk) + 4.38 (team) + 4.68 (process)

= 20.29

B = 0.91 + 0.01 × 20.29 = 1.1129

PM_nominal = 2.5 × (50)^1.1129 = 2.5 × 77.76 = 194.41 PM

EM: PDIF high = 1.29 · PERS high = 0.83 · الباقي = 1.0

PM_adj = 194.41 × (1.29 × 0.83) = 194.41 × 1.07 = 208.16 PM

التحقق: PERS أعلى = جهد أقل (0.83)؛ PDIF أعلى = جهد أكثر (1.29). ✓

مثال محلول: Example 4.11 (سلايد 121–122) — Post-Architecture

المعطيات: نفس حجم Example 4.10 لكن الـ17 driver: RELY high (1.15), DATA very high (1.19), TIME high (1.11), ACAP very high (0.67), PCAP high (0.87)، الباقي nominal.

(PM_nominal = 194.41 نفسه)

EAF (17 drivers) = 1.15 × 1.19 × 1.11 × 0.67 × 0.87 = 0.885

PM_adj = 194.41 × 0.885 = 172.05 PM

⚠ **انتبه: PM_adj هنا (post-arch) أقل** من الـ(172 vs 208) early-design — لأن الـ17 driver مع ACAP very high (0.67) خفّضت أكثر. **الدرس: عدد drivers أكثر ≠ جهد أكثر — يعتمد على القيم.**

1.F — Putnam / Rayleigh Model

مثال محلول: Example 4.12 (سلايد 131–132)

المعطيات: t_d = 1.75 years، K = 95 PY. مطلوب: average rate + peak manning.

$m_0 = K/(t_d \times e) = 95/(1.75 \times 2.71828) = 95/4.757 = 19.97 \approx 20$ persons

Average rate = m_0/t_d = 19.97/1.75 = 11.4 persons/year (≈ 0.95 persons/month)

(ملاحظة: السلايد يطبع m_0=33 مع أرقام متضاربة — الحساب الصحيح أعلاه؛ اعتمد الصيغة $m_0=K/(t_d \cdot e)$.)

مثال محلول: Example 4.13 (سلايد 133–136)

المعطيات: K=600 PY، td=3.5 years.

(a) $m_0 = 600/(3.5 \times 1.648) = 600/5.768 = 104$ persons

peak time = td = 3.5 years

(b) $y(t) = K[1 - e^{(-a \cdot t^2)}]$ حيث $a = 1/(2td^2) = 1/(2\times12.25) = 0.0408$

t = 1.17 years ⇒ $y(1.17) = 600[1 - e^{(-0.0408\times1.369)}] = 600[1 - e^{(-0.0559)}]$

$= 600[1 - 0.9456] = 600 \times 0.0544 = 32.6$ PY

التحقق: بعد ~1/3 من المدة، استُهلك ~5% من الجهد فقط — يتوافق مع شكل منحنى Rayleigh (يبدأ بطيئاً). ✓

مثال محلول: Example 4.14 (سلايد 144–145) — Build-up & Difficulty

$D = K/td^2 = 600/(3.5)^2 = 600/12.25 = 48.98 \approx 49$ person/year

$D_0 = K/td^3 = 600/(3.5)^3 = 600/42.875 = 13.99 \approx 14$ person/year²

التحقق: D_0=14 قريب من 15 ⇒ "new standalone system". ✓

مثال محلول: Example 4.15 (سلايد 157–160) — Development sub-cycle

المعطيات: 90 PY خلال **development sub-cycle**، td = 3.41 years.

(a) yd(td) = 0.95 × Kd = 0.95 × 90 = 85.5 PY

(b) tod = td/√6 = 3.41/2.449 = 1.39 years ≈ 17 months

(c) K = 6 × Kd = 6 × 90 = 540 PY

$D = K/td^2 = 540/(3.41)^2 = 46$ person/year

$D_0 = K/td^3 = 540/(3.41)^3 = 13.6$ person/year²

المفصل الحرج: "development sub-cycle" ⇒ نستعمل Kd و`K=6Kd` و`tod=td/√6` (مو النموذج الكامل).

مثال محلول: Example 4.16 (سلايد 161–163) — استخراج technology factor

المعطيات: 32 PY حتى development cycle، 48000 LOC، td = 25 months = 2.08 years.

Kd = yd/0.95 = 32/0.95 = 33.7 PY

K = 6 × 33.7 = 202 PY

tod = td/√6 = 2.08/2.449 = 0.85 years ≈ 10 months

$D = K/td^2 = 202/(2.08)^2 = 46.7$ person/year

$D_0 = K/td^3 = 202/(2.08)^3 = 22.5$ person/year²

$C = S \cdot K^{(-1/3)} \cdot td^{(-4/3)} = 48000 \times (202)^{(-1/3)} \times (2.08)^{(-4/3)} \approx 3077$

التحقق: C=3077 داخل المدى 610–57314. ✓

مثال محلول: Example 4.17 (سلايد 164–165)

المعطيات: td=1.8 years، C=2400، Kd=25 PY ⇒ K=150 PY.

S = C·K^(1/3)·td^(4/3) = 2400 × (150)^(1/3) × (1.8)^(4/3)

= 2400 × 5.313 × 2.18 = 27,920 LOC

مثال محلول: Example 4.18 (سلايد 166–173) — trade-off وأثر تقليل الوقت

المعطيات: LOC C=2200، D_0=7.5، S=55000.

(a) td = [$(S/C)^3$ / D_0]^(1/7) = [$(55000/2200)^3$ / 7.5]^(1/7)

= [$(25)^3$ / 7.5]^(1/7) = [15625/7.5]^(1/7) = (2083.3)^(1/7) = 3.0 years

K = D_0 × td^3 = 7.5 × 27 = 202.5 PY

D = K/td^2 = 202.5/9 = 22.5 person/year

m_0 = K/(td·e) = 202.5/(3.0×1.648) = 41 persons (approx)

(b) خفّضت td لسنتين و10 أشهر (2.8y):

K = $D_0 \cdot td^3$ = 7.5 × 21.95 = 164.6 PY (جهد كلي أقل! لكن...)

D = 164.6/$(2.8)^2$ = 21 person/year

m_0 = 164.6/(2.8×1.648) = 35.7 persons ← ذروة أعلى بسبب إجهاد الوقت

⇒ الاستنتاج: تقليل الوقت ⇒ manpower build-up أعلى (Do) ⇒ ذروة توظيف أعلى & كلفة أكبر

الدرس (سلايد 170–172): "تقليل الوقت = زيادة حادة بالذروة والكلفة، وإنتاج software أقل". هذا جوهر cost trade-off–time.

المحور 2 — الفهم (Understand)

Q2.1 ليش الـFunction Points أفضل من LOC للغات عالية المستوى؟

> LOCتعاقب اللغات المختصرة: نفس الوظيفة تحتاج 100 سطر بـassembly لكن 10 بـFP. C++ تقيس **الوظيفة** (inputs/outputs/files) بمعزل عن اللغة ⇒ أعدل. *(سلايد 15)*

Q2.2 ليش تقدير الحجم (Size) هو "المدخل" لباقي التقديرات؟

> لأنه لما نعرف الحجم، نشتق منه: cost (عبر rate×efforts)، development time (عبر COCOMO D)، وresources. *(سلايد 33, Fig 1)*

Q2.3 ليش الـCAF يتغيّر من 0.65 إلى 1.35؟

> لأنه `0.65 + ΣFi×0.01`، وΣFi بين 0 (كل الـ14 عامل = 0 = لا تأثير) و70 (كلها = 5 = تأثير أقصى) ⇒ المدى [0.65, 1.35]. *(سلايد 21)*

Q2.4 شنو منطق وجود 3 مودات بـCOCOMO؟

> لأن "الحجم + البيئة" يغيّرون العلاقة بين الجهد والحجم. Organic = بيئة ودّية (a_b=2.4، أقل جهد لنفس الحجم)، Semidetached = وسط (3.0)، Embedded = قيود صارمة على hardware/الجدول (a_b=3.6 ⇒ أعلى جهد). *(سلايد 43, 45)*

Q2.5 ليش Putnam سمّيت "Rayleigh curve model"؟

> لأن Norden لاحظ أن توزيع جهد المشاريع البرمجية يشبه منحنى Rayleigh (احتمالي) — يبدأ بطيئاً، يذروه، ثم ينزل. *(سلايد 123–125)*

المحور 3 — التحليل (Analyze)

Q3.1 من العلاقة التالية، اشتق: كيف يتغير E لو تضاعف الحجم؟

> `E = a_b(KLOC)^b_b`
>
> **الحل:** E'/E = (b_b)^2 = 2^1.05 = 2.07 للـorganic (b=1.05): الجهد يزيد ~107%. للـembedded (b=1.20): 2^1.20 = 2.30 ⇒ +130%. **الاستنتاج:** المودات الأقسى تعاني أكثر من تضاعف الحجم (diseconomies of scale).

Q3.2 من معادلة Rayleigh التالية، أثبت أن الذروة عند t=td:

> `m(t) = (K/td²)·t·e^(-t²/(2td²))`
>
> **الحل:** dm/dt = 0 ⇒ t = td – 1 ⇒ t²/td² = 0. *(سلايد 127)*

Q3.3 ليش $D_0 = K/td^3$ يعطي 8/15/27 للمشاريع الثلاثة النوعية؟

> لأن D_0 يقيس "إجهاد البناء" — مشروع جديد بتفاعلات كثيرة يحتاج build-up أسرع (8، أبطأ نمو)، standalone (15)، إعادة بناء (27، أسرع نمو لأن الأساس موجود). *(سلايد 143)*

Q3.4 حلّل: ليش معامل a بـ**2.4** organic COCOMO لكن بـ**3.2** intermediate؟

> لأن الـintermediate يضرب بـEAF. لو الـEAF=1 (nominal)، الـequation intermediate لازم يعطي نفس النتيجة ⇒ a_i أكبر ليغطي أن EAF سيكون عادة > 1 (لأن العوامل القوية تخفّض الجهد أكثر مما ترفعه). *(سلايد 57)*

Q3.5 اشتق العلاقة التالية من معادلة الإنتاجية:

المطلوب: `S = 0.3935·ϕ·K^(1/3)·td^(4/3)`

المعطى: `P = ϕD^(−2/3)`

الحل: `E = y(td) = K(1−e^(−1/2)) = 0.3935K` نعوّض `P = S/E ⇒ S = P·E = ϕD^(−2/3)·E` و `D = K/td²`:

`S = ϕ(K/td²)^(−2/3)·(0.3935K) = ϕ·K^(−2/3)·td^(4/3)·(0.3935K) = 0.3935·ϕ·K^(1/3)·td^(4/3)`. *(سلايد 147)*

المحور 4 — التقييم (Evaluate)

Q4.1 أي مديل تقدير تختار لمشروع صغير بتقنية مألوفة، ولماذا؟

Basic COCOMO (organic) أو **FP**. صغير + مألوف ⇒ العلاقات بسيطة، Basic كافي؛ FP يعطيك حجم مستقل عن اللغة. **لا** تختار Detailed/Putnam (مبالغة).

Q4.2 قارن Walston-Felix وSEL — أيها أكثر تحفظاً؟

W-F أكثر تحفظاً: a=5.2 (أكبر) ⇒ لنفس الجهد، LOC أقل (94K vs 24K). SEL أتفاؤل. **التوصية:** استخدم SEL للأنظمة الكبيرة المُختبَرة، W-F للمحافظة.

Q4.3 هل "تسريع المشروع" فكرة جيدة؟ برّر بـPutnam.

لا غالباً. من العلاقة `D = K/td²` (ومثال 4.18): تقليل td يرفع K (الجهد الكلي) و m_0 (الذروة)، وقد ينخفض الـsoftware المُنتَج. **Brooks' Law** داعم [معياري].

Q4.4 أي مرحلة COCOMO II الأكثر ملاءمةً للمشاريع المتوسطة؟

Early Design — توازن بين الدقة (7 drivers + 5 scale factors) والسرعة قبل اكتمال الـarchitecture.

المحور 5 — التصميم/الإنشاء (Create)

Q5.1 أعطيتك مشروع: 25 KLOC، بيئة معقّدة (قيود صارمة على hardware)، موثوقية عالية، قدرة محلل ومبرمج عالية. قدّر الكلفة والجدول + اقترح خطة.

الخطوة 1: اختر الموديل: قيود صارمة ⇒ **embedded**، لكن الـdrivers تحتاج intermediate.

الخطوة 2: احسب الجهد بالمعادلة:

```
E = 2.8 × (25)^1.20 × EAF
```

حيث EAF من RELY (عالية، 1.15) × ACAP (عالية، 0.86) × PCAP (عالية، 0.87).

الخطوة 3: احسب المدة وحجم الفريق:

```
D = 2.5 × E^0.32 · SS = E / D
```

الخطوة 4: خطة: team build-up حسب Rayleigh، مراحل مع جهد phase-wise.

Q5.2 صمّم RMMM (Risk Mitigation, Monitoring, Management) plan لمشروع يتوظف مطوّرين مبتدئين.

خطة: **Mitigation** (تدريب مكثّف قبل البدء، pairing) · **Monitoring** (تتبع الأخطاء أسبوعياً، code review) · **Management** (خطة بديلة لمبرمج أساسي، احتياطي وقت). *(خارجي/pressman-style — من Planning_References_03)*

Q5.3 اقترح استراتيجية تقدير لمشروع تتغيّر متطلباته بسرعة.

لا تعتمد LOC (يتغير). استخدم **FP** (يُعاد حسابها مع كل تغيير متطلبات) + **incremental estimation** + هامش احتياطي contingency 30-50%).

✅ قائمة التحقق (Self-check)

- [ ] أحسب FP لمشروع بتعقيدات مختلطة بدون خطأ
- [ ] أختار الموديل الصحيح من وصف الحجم/البيئة
- [ ] أحسب E وD لكل المودات الثلاثة
- [ ] أحسب EAF من drivers
- [ ] أحسب B وPM لـCOCOMO II
- [ ] أحسب Kd ,D_0, D, m_0, Putnam tod
- [ ] أشتق S من $P=\varphi D^{(-2/3)}$
- [ ] أشرح trade-off وكيف أحسب أثره

Retrieval set (active recall)

السؤال	الجواب
خطوات حل أي مسألة FP؟	عدّ units ← أوزان ← UFP ← CAF ← FP
المعادلة الأساسية basic COCOMO؟	E = a_b(KLOC)^b_b
متى نستخدم a_i بدل a_b؟	لما فيه (intermediate) cost drivers
كيف نحسب B بـCOCOMO II؟	B = 0.91 + 0.01×ΣSF

السؤال	الجواب
كيف نحسب m_0؟	K/(td·e)
كيف نحسب D_0؟	K/td^3
متى نستخدم Kd وtod؟	بالـsub-cycle development فقط
كيف نشتق S من K وtd؟	S = 0.3935·φ·K^(1/3)·td^(4/3)

المصدر: سلايدات 1–207 + الأمثلة 4.1–4.19 — بُني 12-10-2026